

Assignment Documentation

Factory Inventory & Production Management System (MVP) Start up manager

This document records the startup-manager assignment deliverables derived from the implemented monorepo, sprint records, and the current deployment runbooks.

1. Project Plan

1.1 Project Overview

The project is a single-factory food manufacturing inventory and production management system built as a TypeScript monorepo. The frontend is a React + Vite single-page application and the backend is an Express API using Firebase Authentication, MongoDB Atlas, RBAC, audit logging, stock movement tracking, FEFO dispatch allocation, and delivery-note PDF generation.

1.2 Project Objectives

- Provide one system for owner, employee, and wholesaler users with server-side role enforcement.
- Control raw-material stock-in, production batch completion, and finished-goods dispatch with traceability.
- Maintain an auditable stock ledger and current stock snapshots for operational reporting.
- Support practical deployment on Vercel with Firebase and MongoDB Atlas.
- Deliver a demo-ready MVP with clean setup, seed data, and role-based documentation.

1.3 Scope

- Included: authentication, users, products, suppliers, wholesalers, GRN, BOM, batches, orders, dispatch, reports, audit, deployment/runbooks, and demo data seeding.
- Included: delivery-note PDF generation, low-stock and expiry reporting, and FEFO stock allocation.
- Excluded from MVP scope: finance/accounting, advanced analytics, credit workflows, and multi-factory support.

1.4 Major Modules and Features

- Access control: Firebase sign-in on client, ID-token verification on server, Mongo user-role mapping, and protected routes.
- Master data: products, suppliers, wholesalers, BOM records, and owner-managed users.
- Operations: GRN draft/confirm, batch create/start/complete, wholesaler orders, dispatch creation, and delivery-note PDFs.
- Visibility: stock-on-hand, movements, expiry, production, wastage, sales-by-wholesaler, traceability, and audit logs.
- Deployment foundation: Vercel client/server split, serverless API route mapping, CORS allowlist, env validation, and seed data.

1.5 Project Products and Deliverables

Deliverable	Repository Evidence	Purpose
Frontend SPA	client/src/app, client/src/pages	Role-based UI for operations, reports, and administration.

Deliverable	Repository Evidence	Purpose
Backend API	server/src/app.ts and server/src/modules/*	Validated REST endpoints with RBAC and audit logging.
Persistence Layer	server/src/db/models/*	MongoDB-backed operational and master data.
Automation and Seed	server/src/seed.ts, tests	Demo readiness, repeatable setup, and regression coverage.
Technical Documentation	README.md, docs/*.md, role folders	Setup, deployment, user guidance, architecture, and sprint evidence.

1.6 Team Responsibility Summary

Role	Primary Assignment Responsibility
Start up manager	Overall technical foundation, architecture bootstrap, main implementation outside delegated ownership areas, deployment hardening, and integration closure.
Project manager	Project coordination, authentication-related ownership, requirement tracking, and review decisions.
Scheduling manager	Schedule/progress management, communication cadence, and product CRUD workstream coordination.
Quality manager	Supplier-operation quality checks, validation, QA reporting, and acceptance evidence.
Risk manager	Risk identification, mitigation planning, delivery blockers, and reporting-related operational risks.

2. Team Code of Conduct (Derived from BCS Principles)

The following conduct rules are derived from the BCS code of conduct and adapted to this project team as practical rules for day-to-day delivery.

2.1 Public Interest and User Impact

- We avoid changes that could compromise inventory accuracy, traceability, or delivery records for the factory business.
- We do not expose secrets, private user data, or service-account credentials in code, commits, screenshots, or documents.
- We document known risks and limitations instead of hiding them during demonstrations or submissions.

2.2 Professional Competence and Integrity

- We base implementation and documentation on the real repository state, not on invented features.
- We test important flows before claiming completion, especially authentication, stock movements, batches, dispatch, and reports.
- We record assumptions clearly when reverse engineering sprint and role evidence.

2.3 Duty to Relevant Authority

- We keep sprint records, review notes, and deployment instructions aligned with the system that actually runs.
- We escalate blockers such as CORS failures, route mismatches, or broken verification commands early rather than late.
- We preserve repository structure and auditability so supervisors can review evidence directly.

2.4 Duty to the Profession and Team Practice

- We review each other's work respectfully and focus on defects, risks, and evidence-based improvement.
- We keep changes minimal, consistent, and maintainable, especially near submission deadlines.
- We use version-controlled documentation and avoid deleting project evidence without a clear reason.

2.5 Practical Conduct Rules for This Project

- Before merging technical changes, confirm impact on auth, inventory integrity, dispatch, reports, and deployment configuration.
- Before recording or sharing demos, remove real secrets from env files and verify only placeholder values appear in documentation.
- When a production defect is found, first reproduce it, record the actual failing route or UI, then fix the narrowest safe cause.
- All team members maintain truthful role-based records so the final submission remains academically defensible.

3. Conclusion

The startup-manager assignment deliverables are satisfied through a concrete project plan grounded in the implemented monorepo and a project-specific team code of conduct based on BCS principles.